

Technology Skill Development Lab Java

Assignment No.5

Student Name : Saurabh Ashok Wagh

Roll No. : 26

Division : A

Batch : A2

Class : SY

Branch : B.Tech AIDS

Course Teacher Name : Prof. Vishakha Salunkhe

Problem Statement : Create a base class Account featuring methods deposit() and withdraw(). These methods should be overridden by derived classes to execute specific actions tailored to different account types.

a. Method Overloading: Derive a class SavingsAccount from Account. Implement method overloading for withdraw() i.Illustrate polymorphism by invoking the withdraw() method on both objects, highlighting distinct behaviors based on the account type. by introducing an additional parameter to specify the reason for withdrawal.

Create a Java program using method overloading to implement a Data Categorization Algorithm.

b. Method Overriding: Derive a class CheckingAccount from Account. Implement method overriding for withdraw() within the CheckingAccount class, introducing overdraft protection.

c. Polymorphic Demonstration:

i) Instantiate objects for both SavingsAccount and CheckingAccount.

ii) Illustrate polymorphism by invoking the withdraw() method on both objects, highlighting distinct behaviors based on the account type.

Develop a Java program using inheritance to demonstrate method overriding. Create a base class Account and a derived class SavingsAccount that overrides the withdraw() method. Implement a Rule-Based Classification Algorithm to categorize transactions as “Approved” or “Denied” based on balance and minimum balance rules.

Program:

```
class Account {
    double balance;

    Account(double balance) {
        this.balance = balance;
```

```
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount deposited: " + amount);
    }

    void withdraw(double amount) {
        balance = balance - amount;
        System.out.println("Amount withdrawn: " + amount);
    }
}


class SavingsAccount extends Account {

    SavingsAccount(double balance) {
        super(balance);
    }

    @Override
    void withdraw(double amount) {
        if (balance - amount >= 500) {
            balance = balance - amount;
            System.out.println("Savings Account: Withdrawal successful");
        } else {
            System.out.println("Savings Account: Withdrawal denied");
```

```
        }
    }

    void withdraw(double amount, String reason) {
        System.out.println("Reason: " + reason);
        withdraw(amount);
    }
}


class CheckingAccount extends Account {

    double overdraftLimit = 1000;

    CheckingAccount(double balance) {
        super(balance);
    }

    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Checking Account: Withdrawal successful");
        } else {
            System.out.println("Checking Account: Withdrawal denied");
        }
    }
```

```
}

class DataCategorization {

    void categorize(int data) {
        System.out.println(data + " -> Integer");
    }

    void categorize(String data) {
        System.out.println(data + " -> String");
    }

    void categorize(double data) {
        System.out.println(data + " -> Decimal");
    }
}

class Transaction {

    void checkTransaction(double balance, double amount) {

        double minimumBalance = 500;

        if (balance - amount >= minimumBalance) {
            System.out.println("Transaction: Approved");
```

```
        } else {
            System.out.println("Transaction: Denied");
        }
    }
}


public class Main {
    public static void main(String[] args) {

        SavingsAccount savings = new SavingsAccount(5000);

        savings.deposit(1000);
        savings.withdraw(2000);
        savings.withdraw(500, "Education");

        System.out.println();


        CheckingAccount checking = new CheckingAccount(2000);

        checking.deposit(500);
        checking.withdraw(3000);

        System.out.println();
```

```
        Account a1 = new SavingsAccount(4000);
        Account a2 = new CheckingAccount(2000);

        a1.withdraw(3000);
        a2.withdraw(2500);

        System.out.println();


        DataCategorization data = new DataCategorization();

        data.categorize(10);
        data.categorize("Hello");
        data.categorize(10.5);

        System.out.println();


        Transaction t = new Transaction();

        t.checkTransaction(5000, 3000);
        t.checkTransaction(2000, 1800);
    }
}
```

Output :

```
Amount deposited: 1000.0
Savings Account: Withdrawal successful
Reason: Education
Savings Account: Withdrawal successful

Amount deposited: 500.0
Checking Account: Withdrawal successful

Savings Account: Withdrawal successful
Checking Account: Withdrawal successful

10 -> Integer
Hello -> String
10.5 -> Decimal

Transaction: Approved
Transaction: Denied
```

Conclusion: This Java program demonstrates the concepts of **inheritance, method overloading, method overriding, and polymorphism**. The `SavingsAccount` class uses a minimum balance rule, while the `CheckingAccount` provides overdraft protection. The program also categorizes withdrawals as **Approved or Denied** based on predefined rules. Thus, it shows how inheritance and polymorphism can be used to implement different behaviors for different types of accounts.